

Lab 7: Thực hành thuật toán SHA-256

ThS. Lê Thị Thùy Trang

2026-5-14

1. Giới thiệu về bài thực hành

Trong bài thực hành này, sinh viên phân tích và mở rộng chương trình C++ mô phỏng thuật toán SHA-256. Khác với bài AES tập trung vào mã hóa/giải mã, bài SHA tập trung vào hàm băm một chiều, kiểm tra toàn vẹn dữ liệu và lưu trữ mật khẩu ở mức minh họa học thuật.

Sau bài thực hành, sinh viên có thể:

- Phân tích được cấu trúc chương trình C++ triển khai thuật toán SHA-256;
- Giải thích được các bước chính của SHA-256: padding, chia khối, tạo message schedule, vòng nén và sinh digest;
- Biên dịch, chạy và kiểm thử chương trình SHA-256 từ dòng lệnh;
- Ứng dụng SHA-256 để kiểm tra tính toàn vẹn của tệp tin;
- Mô phỏng chức năng băm mật khẩu và cải tiến bằng salt;
- Ghi lại minh chứng thực hành thông qua test, log và báo cáo ngắn.

Sinh viên nhận repo starter kit từ GitHub Classroom hoặc từ giảng viên, sau đó lưu về máy.

Cấu trúc repo như sau:

```
fit4012_lab7_sha
├── .github/
│   ├── scripts/
│   │   └── check_submission.sh
│   └── workflows/
│       └── ci.yml
├── docs/
│   └── sha-code-notes.md
├── logs/
│   ├── README.md
│   └── sample-run.log
├── scripts/
│   └── run_sample.sh
├── tests/
│   ├── test_sha_compile.sh
│   ├── test_known_vectors.sh
│   ├── test_file_integrity_tamper.sh
│   ├── test_password_hash.sh
│   └── test_salted_password.sh
├── CMakeLists.txt
└── Makefile
```

```
README.md
file_integrity.cpp
password_hash.cpp
report-1page.md
salted_password_hash.cpp
sha256_lib.h
sha_procedure.cpp
structure.h
```

2. Nhắc lại lý thuyết

SHA-256 là một hàm băm mật mã, nhận đầu vào có độ dài bất kỳ và tạo ra giá trị băm có độ dài cố định 256 bit, thường được biểu diễn bằng chuỗi hexa 64 ký tự.

Thuật toán SHA-256 có một số đặc điểm quan trọng:

- **Một chiều:** từ giá trị băm rất khó suy ngược lại dữ liệu gốc;
- **Nhạy với thay đổi nhỏ:** chỉ cần thay đổi một ký tự trong dữ liệu đầu vào, giá trị băm có thể thay đổi hoàn toàn;
- **Độ dài đầu ra cố định:** dù dữ liệu đầu vào ngắn hay dài, digest SHA-256 vẫn có độ dài 256 bit;
- **Phù hợp để kiểm tra toàn vẹn:** có thể phát hiện tệp tin bị sửa đổi bằng cách so sánh hash trước và sau;
- **Không phải giải pháp lưu mật khẩu đầy đủ cho hệ thống thật:** nếu dùng cho mật khẩu cần có salt, cơ chế làm chậm và thuật toán chuyên dụng hơn như bcrypt, scrypt hoặc Argon2id.

2.1 Quy trình tổng quát của SHA-256

Quy trình SHA-256 có thể tóm tắt như sau:

```
Message
↓
Padding
↓
Chia thành các block 512 bit
↓
Với mỗi block:
- Tạo W[0..63]
- Khởi tạo a, b, c, d, e, f, g, h
- Chạy 64 vòng nén
- Cập nhật state H[0..7]
↓
Ghép H[0..7] thành digest 256 bit
↓
Chuỗi hexa 64 ký tự
```

2.2 Padding trong SHA-256

SHA-256 không dùng zero padding như một số ví dụ mã hóa khối nhập môn. Thuật toán dùng padding chuẩn theo cấu trúc Merkle-Damgård:

1. Thêm byte '0x80', tương ứng với một bit '1' theo sau là các bit '0'.

2. Thêm các byte '0x00' cho đến khi độ dài dữ liệu đạt '56 mod 64' byte.
 3. Thêm độ dài ban đầu của thông điệp dưới dạng số nguyên '64 bit' theo thứ tự Big Endian.
- Nhờ bước padding, dữ liệu đầu vào luôn có thể được chia thành các khối 512 bit, tương đương 64 byte.

2.3 Các thành phần chính trong SHA-256

Trong repo này, sinh viên cần chú ý các thành phần sau:

- 'structure.h': chứa các hằng số và hàm logic cơ bản của SHA-256.
- sha256_lib.h: đóng gói các hàm xử lý SHA-256 để có thể dùng lại trong nhiều chương trình.
- sha_procedure.cpp: chương trình chính để chạy self-test, băm chuỗi và băm file.
- file_integrity.cpp: ứng dụng SHA-256 để kiểm tra tệp tin có bị thay đổi hay không.
- password_hash.cpp: minh họa băm mật khẩu và kiểm tra mật khẩu đúng/sai.
- salted_password_hash.cpp: cải tiến bằng salt để cùng một mật khẩu không tạo ra cùng một bản ghi hash.

3. Hướng dẫn chuẩn bị môi trường

3.1 Yêu cầu phần mềm

Sinh viên cần có:

- Trình biên dịch g++ hỗ trợ C++17;
- Công cụ make nếu chạy bằng Makefile;
- Git nếu clone repo từ GitHub Classroom;
- VSCode hoặc IDE tương đương để đọc và chỉnh sửa mã nguồn.

Kiểm tra phiên bản g++:

```
g++ --version
```

3.2 Tải repo về máy

Sinh viên bấm link bài tập, accept assignment, sau đó clone repo cá nhân về máy:

```
git clone <link_repo_cua_sinh_vien>
cd fit4012_lab7_sha
```

4. Hướng dẫn thực hiện

4.1 Biên dịch chương trình

Cách khuyến nghị là dùng Makefile:

```
make
```

Sau khi biên dịch, repo tạo ra 4 chương trình:

```
sha256
file_integrity
password_hash
salted_password_hash
```

Sinh viên cũng có thể biên dịch trực tiếp từng file:

```
g++ -std=c++17 -Wall -Wextra -pedantic sha_procedure.cpp -o sha256
g++ -std=c++17 -Wall -Wextra -pedantic file_integrity.cpp -o file_integrity
g++ -std=c++17 -Wall -Wextra -pedantic password_hash.cpp -o password_hash
g++ -std=c++17 -Wall -Wextra -pedantic salted_password_hash.cpp -o salted_password_hash
```

4.2 Chạy kiểm thử nhanh thuật toán SHA-256

Chạy self-test để kiểm tra các test vector cơ bản:

```
./sha256 --self-test
```

Chạy băm chuỗi abc:

```
./sha256 --hash-string "abc"
```

Kết quả SHA-256 chuẩn của chuỗi abc là:

```
ba7816bf8f01cfea414140de5dae2223
b00361a396177a9cb410ff61f20015ad
```

Do chuỗi hash dài 64 ký tự, khi ghi vào báo cáo sinh viên có thể để trên một dòng hoặc tách thành hai dòng cho dễ nhìn.

4.3 Chạy script mẫu

Repo đã chuẩn bị script mẫu để sinh viên quan sát toàn bộ luồng chạy:

```
bash scripts/run_sample.sh
```

Script này minh họa các thao tác:

- Chạy self-test;
- Băm chuỗi;
- Băm file;
- Kiểm tra file trước và sau khi bị sửa;
- Băm mật khẩu;
- Băm mật khẩu có salt.

4.4 Chạy toàn bộ test

Chạy toàn bộ test bằng lệnh:

```
make test
```

Các test chính gồm:

- `test_sha_compile.sh`: kiểm tra chương trình có biên dịch được không;

- `test_known_vectors.sh`: kiểm tra giá trị hash với test vector chuẩn;
- `test_file_integrity_tamper.sh`: kiểm tra khả năng phát hiện file bị sửa;
- `test_password_hash.sh`: kiểm tra mật khẩu đúng/sai;
- `test_saltd_password.sh`: kiểm tra cùng mật khẩu nhưng salt khác nhau tạo ra hash khác nhau.

5. Nhiệm vụ thực hành

5.1 Q1 - Chạy và phân tích quy trình SHA-256

Sinh viên thực hiện:

1. Biên dịch chương trình bằng ‘make’.
2. Chạy self-test:
““ bash ./sha256 --self-test ““
3. Bấm ít nhất 3 chuỗi khác nhau:
““ bash ./sha256 --hash-string "abc" ./sha256 --hash-string "FIT4012" ./sha256 --hash-string "FIT4012!" ““
4. So sánh hash của ‘FIT4012’ và ‘FIT4012!’ để quan sát hiệu ứng thay đổi nhỏ trong đầu vào.
5. Mô tả lại quy trình SHA-256 bằng lời của mình trong ‘report-1page.md’.

Gợi ý trả lời:

- Dữ liệu đầu vào được padding như thế nào?
- Vì sao cần chia thành block 512 bit?
- $W[0..63]$ có vai trò gì?
- Vì sao output luôn dài 64 ký tự hexa?

5.2 Q2 - Kiểm tra toàn vẹn file

Mục tiêu của câu này là dùng SHA-256 để phát hiện file có bị thay đổi hay không.

Tạo một file mẫu:

```
printf "FIT4012 SHA sample\n" > sample.txt
```

Tính hash của file:

```
./sha256 --hash-file sample.txt
```

Dùng chương trình `file_integrity` để kiểm tra file với hash kỳ vọng:

```
./file_integrity sample.txt <expected_hash>
```

Sau đó sửa nội dung file:

```
printf "tampered\n" >> sample.txt
```

Chạy lại chương trình kiểm tra:

```
./file_integrity sample.txt <expected_hash>
```

Sinh viên cần ghi lại:

- Hash ban đầu của file;
- Kết quả kiểm tra trước khi sửa file;
- Kết quả kiểm tra sau khi sửa file;
- Nhận xét vì sao SHA-256 giúp phát hiện thay đổi dữ liệu.

5.3 Q3 - Băm và kiểm tra mật khẩu

Mục tiêu của câu này là mô phỏng quá trình lưu mật khẩu ở dạng hash thay vì lưu plaintext.

Ví dụ đăng ký mật khẩu:

```
./password_hash register password.hash
```

Ví dụ đăng nhập/kiểm tra mật khẩu:

```
./password_hash verify password.hash
```

Sinh viên cần kiểm thử ít nhất 2 trường hợp:

- Nhập đúng mật khẩu đã đăng ký;
- Nhập sai mật khẩu.

Yêu cầu ghi vào logs/:

- Log khi đăng ký mật khẩu;
- Log khi xác thực đúng;
- Log khi xác thực sai.

Lưu ý: chương trình này chỉ minh họa nguyên lý. Trong hệ thống thật, không nên chỉ dùng SHA-256 đơn thuần để lưu mật khẩu vì kẻ tấn công có thể thử rất nhiều mật khẩu với tốc độ cao.

5.4 Q4 - Băm mật khẩu với salt

Mục tiêu của câu này là tránh tình huống hai người dùng có cùng mật khẩu thì có cùng hash.

Salt là một chuỗi ngẫu nhiên được thêm vào mật khẩu trước khi băm. Salt không cần giữ bí mật, nhưng cần khác nhau giữa các người dùng hoặc giữa các lần đăng ký.

Ví dụ đăng ký mật khẩu có salt:

```
./salted_password_hash register salted_password.hash
```

Ví dụ xác thực mật khẩu:

```
./salted_password_hash verify salted_password.hash
```

Định dạng file lưu trữ trong repo là:

```
salt:hash
```

Sinh viên cần chứng minh:

- Cùng một mật khẩu nhưng đăng ký hai lần sẽ tạo ra hai bản ghi salt:hash khác nhau;
- Khi xác thực, chương trình đọc lại salt, băm lại salt + password, rồi so sánh với hash đã lưu;
- Salt làm giảm rủi ro từ bảng hash có sẵn như rainbow table.

6. Ghi log và minh chứng

Thư mục `logs/` dùng để lưu minh chứng thực hành. Sinh viên có thể ghi log bằng cách copy output terminal hoặc dùng redirect.

Ví dụ:

```
make test > logs/test-result.log 2>&1
bash scripts/run_sample.sh > logs/run-sample-my-machine.log 2>&1
```

Mỗi nhóm/cá nhân cần có ít nhất 1 file log thật trong thư mục `logs/`. Log nên thể hiện được:

- Chương trình biên dịch thành công;
- Test vector SHA-256 chạy đúng;
- Kiểm tra file bị sửa cho kết quả [FAIL];
- Kiểm tra mật khẩu đúng/sai;
- Salt tạo ra bản ghi khác nhau cho cùng một mật khẩu.

7. Báo cáo và yêu cầu nộp bài

Sinh viên hoàn thiện file `report-1page.md` trong repo. Báo cáo nên ngắn gọn, tập trung vào minh chứng và nhận xét.

Nội dung tối thiểu gồm:

1. **Mục tiêu:** nêu mục tiêu của bài thực hành.
2. **Cách làm:** mô tả ngắn cách chạy SHA-256, kiểm tra file, băm mật khẩu và thêm salt.
3. **Kết quả:** ghi lại hash của chuỗi/file, kết quả kiểm tra file, kết quả xác thực mật khẩu.
4. **Kết luận:** rút ra vai trò của SHA-256 trong kiểm tra toàn vẹn và hạn chế khi dùng cho mật khẩu.

Trước khi nộp, sinh viên chạy:

```
make test
bash .github/scripts/check_submission.sh
```

Nếu cả hai lệnh đều pass, repo đã đáp ứng yêu cầu kiểm tra cơ bản.

8. Checklist nộp bài

Sinh viên kiểm tra các mục sau trước khi submit:

- Chạy được ‘make’ không lỗi.
- Chạy được ‘make test’ không lỗi.
- Hoàn thiện ‘report-1page.md’.
- Có ít nhất 1 file log thật trong thư mục ‘logs/’.
- Có minh chứng kiểm tra file bị sửa/tamper.
- Có minh chứng xác thực mật khẩu đúng/sai.
- Có minh chứng cùng mật khẩu nhưng salt khác nhau tạo ra hash khác nhau.

- Không xóa các file test bắt buộc trong thư mục 'tests/'.

9. Rubric

Tiêu chí	Mô tả	Điểm
Biên dịch và chạy chương trình	Repo biên dịch được, chạy được sha256, file_integrity, password_hash, salted_password_hash.	2.0
Phân tích SHA-256	Giải thích được padding, block 512 bit, message schedule, 64 vòng nén và digest 256 bit.	2.0
Kiểm tra toàn vẹn file	Tính hash file, so sánh với hash kỳ vọng, phát hiện được file bị sửa.	2.0
Băm mật khẩu và salt	Mô phỏng được đăng ký/xác thực mật khẩu; giải thích được vai trò của salt.	2.0
Minh chứng và báo cáo	Có log, test, báo cáo ngắn rõ ràng, trung thực học thuật.	2.0

10. Lưu ý an toàn và đạo đức học thuật

- Repo này chỉ phục vụ mục tiêu học tập và minh họa thuật toán băm.
- Không dùng bài lab để xử lý hoặc công bố dữ liệu nhạy cảm thật.
- Không trình bày chương trình demo là hệ thống lưu mật khẩu an toàn cho môi trường sản xuất.
- Nếu sử dụng tài liệu, mã nguồn, công cụ hoặc AI để hỗ trợ, sinh viên cần ghi rõ trong báo cáo.
- Khi làm việc nhóm, cần mô tả trung thực phần việc của từng thành viên.